Import Libraries

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
```

Load Dataset

```
df = sns.load_dataset("iris")
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Basic Information

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Check Missing Values

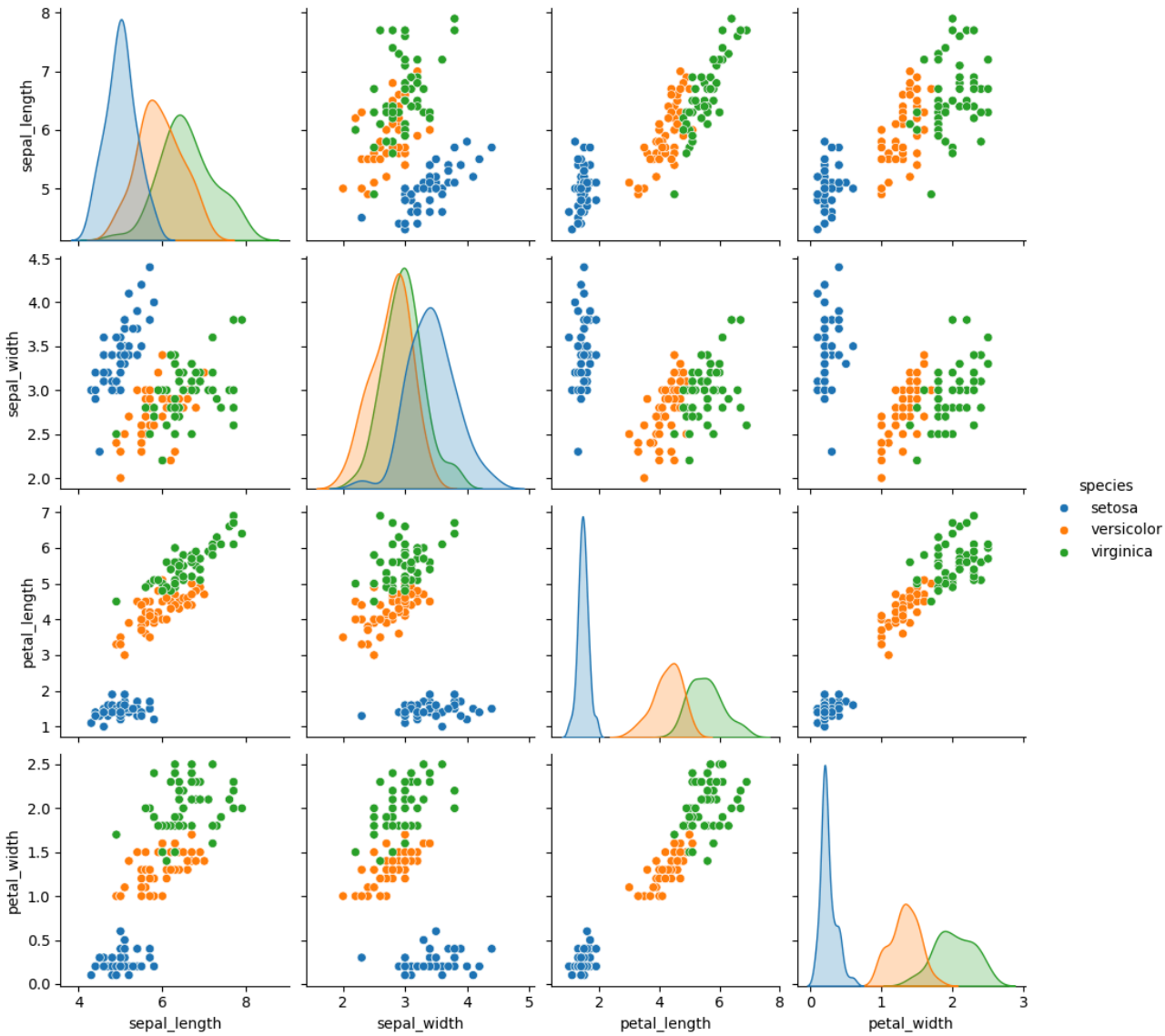
```
df.isnull().sum()
```

	0
sepal_length	0
sepal_width	0
petal_length	0
petal_width	0
species	0

dtype: int64

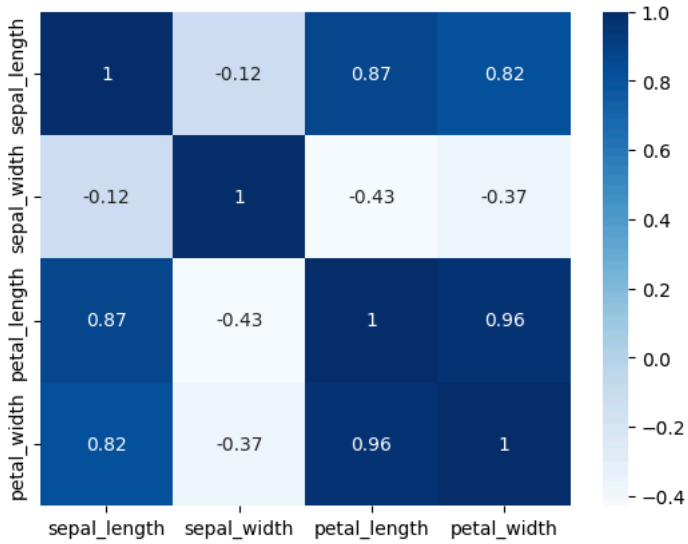
Pairplot

```
sns.pairplot(df, hue="species")
plt.show()
```



Correlation Heatmap

```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="Blues")
plt.show()
```



Features & Target

```
X = df.drop("species", axis=1)
y = df["species"]

X.head(), y.head()
```

```
(   sepal_length  sepal_width  petal_length  petal_width
 0           5.1          3.5           1.4          0.2
 1           4.9          3.0           1.4          0.2
 2           4.7          3.2           1.3          0.2
 3           4.6          3.1           1.5          0.2
 4           5.0          3.6           1.4          0.2,
 0    setosa
 1    setosa
 2    setosa
 3    setosa
 4    setosa
 Name: species, dtype: object)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Feature Scaling

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Build KNN Model

```
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train_scaled, y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

Predictions

```
y_pred = model.predict(X_test_scaled)
y_pred[:5]
```

```
array(['versicolor', 'setosa', 'virginica', 'versicolor', 'versicolor'],
      dtype=object)
```

Evaluation

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

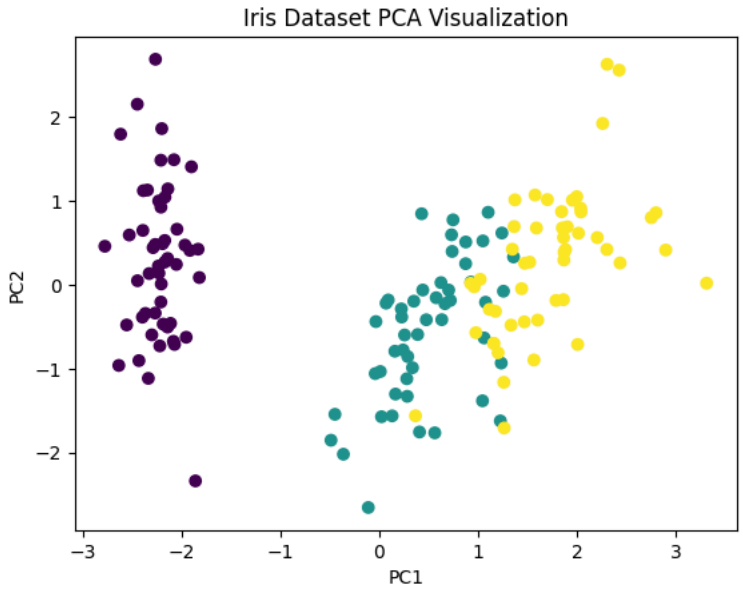
Plot PCA scatter plot

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Scale the entire dataset X for PCA
scaler_full = StandardScaler()
X_scaled_full = scaler_full.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled_full)

# Plot the PCA result
plt.scatter(X_pca[:,0], X_pca[:,1], c=y.astype('category').cat.codes)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Iris Dataset PCA Visualization")
plt.show()
```



Print Explained Variance

```
print(pca.explained_variance_ratio_)
```

```
[0.72962445 0.22850762]
```

Try different K values

```
for k in range(1, 11):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    print("K =", k, "Accuracy =", accuracy_score(y_test, y_pred))
```

```
K = 1 Accuracy = 1.0
K = 2 Accuracy = 1.0
K = 3 Accuracy = 1.0
K = 4 Accuracy = 1.0
K = 5 Accuracy = 1.0
K = 6 Accuracy = 1.0
K = 7 Accuracy = 0.9666666666666667
K = 8 Accuracy = 1.0
K = 9 Accuracy = 1.0
K = 10 Accuracy = 1.0
```